

Stratus - Dokumentacija Koda

Verzija: 1.0.0 | Stack: React 19 + TypeScript · FastAPI + Python 3.12+ · XGBoost (ML) · Open-Meteo API

1. Backend

Backend je Python FastAPI aplikacija koja prima podatke o letu i vremenskim uslovima, pokreće ih kroz trenirani ML model i vraća strukturiranu procjenu kašnjenja. Ulazna tačka je `backend/app/main.py`.

`app/main.py` | `app/routes/health.py`

`main.py` inicijalizira aplikaciju, konfigurira CORS i registrira tri rutera pod prefiksom `/api`. `health.py` izlaže `GET /api/health → {"status": "ok"}` za provjeru dostupnosti servera.

`app/routes/delay.py`

Definiše `POST /api/delay`. `DelayRequest` je Pydantic model s `Field` validacijom za sva polja. `DelayResponse` vraća: `delay_class` (1-4), `label`, `color`, `confidence`, `probabilities` i `recommendation`. Handler delegira na `predict_delay()`.

Polje	Tip	Ograničenja
<code>airline / origin / dest</code>	<code>str</code>	BTS kod (2-3) / IATA kod (3) / IATA kod (3)
<code>month / day_of_week / dep_hour</code>	<code>int</code>	1-12 / 1-7 (1 = ponedjeljak) / 0-23
<code>distance</code>	<code>float</code>	0-10000 km
<code>temperature / precipitation / snowfall</code>	<code>float</code>	-50-60 °C / 0-200 mm/h / 0-100 cm/h
<code>wind_speed / cloud_cover</code>	<code>float</code>	0-250 km/h / 0-100 %

`app/routes/weather.py`

Definiše `GET /api/weather?airport=&date=&hour=`. Validira IATA kod prema `AIRPORTS`, poziva `fetch_forecast()` i vraća pet vremenskih varijabli za odabrani sat. Koristi ga frontend dugme "Preuzmi prognozu"; vraća 404 ako je datum izvan horizonta prognoze (~16 dana).

`ml/airports.py` | `ml/weather.py`

`airports.py` sadrži `AIRPORTS` (50 najprometnijih aerodroma u SAD s koordinatama i IANA vremenskom zonom) i `AIRLINES` (21 BTS kod aviokompanije). `weather.py` je pristupni sloj za Open-Meteo: `fetch_archive()` vraća satne historijske podatke (ERA5) u lokalnom vremenu aerodroma, `fetch_forecast()` vraća prognozu za jedan sat. Obje funkcije proizvode istih pet varijabli (`WEATHER_FEATURES`) kako bi trening i inferencija koristili identičan kod.

`ml/download_data.py`

Preuzima mjesečne zip arhive BTS *On-Time Performance* baze sa javnog URL-a (bez naloga), čita 11 potrebnih kolona, filtrira letove na poznate aerodrome/aviokompanije i snima `ml/data/flights.csv`.

Zatim za svaki aerodrom preuzima satno vrijeme za isti period i snima `ml/data/weather.csv`. Godina i mjeseci se podešavaju varijablama `STRATUS_YEAR / STRATUS_MONTHS`.

ml/preprocessing.py

Definira shemu karakteristika (9 numeričkih + 3 kategorijske) i gradi sklearn `ColumnTransformer`: numerički pod-pipeline (`SimpleImputer` medijan → `StandardScaler`) i kategorički (`SimpleImputer` → `OneHotEncoder` s `handle_unknown="ignore"`). `delay_class()` mapira `ArrDelayMinutes`, `Cancelled` i `Diverted` u klase 1-4 (pragovi 15 i 60 minuta). Konstanta `MILES_TO_KM` služi za pretvaranje BTS udaljenosti iz milja u kilometre.

ml/train_real.py

Skripta za treniranje (`python -m ml.train_real`). `load_and_prepare()` mapira BTS kolone na nazive karakteristika, pretvara udaljenost iz milja u kilometre, izvlači sat polaska iz `CRSDepTime` (hhmm), spaja satno vrijeme po (aerodrom, datum+sat) i čisti podatke. `train()` uzima stratifikovani uzorak do `STRATUS_MAX_ROWS`, dijeli podatke 80/20, računa blage težine klasa ($1/\sqrt{\text{frekvencija}}$), gradi pipeline(preprocesor, `XGBClassifier`) i snima ga u `delay_model.joblib`. Opcija `--synthetic` koristi podatke iz `ml/generate_data.py`.

ml/predict.py

`_load_model()` učitava i kešira `delay_model.joblib`. `predict_delay(FlightInput)` kreira jednoredni `DataFrame`, poziva `predict()` (klasa 0-bazirana, +1 → 1-4), `predict_proba()` (zaokruženo na jednu decimalu), traži preporuku u `RECOMMENDATIONS` i vraća `DelayResult` `dataclass`.

2. Frontend

React 19 + TypeScript SPA izgrađena s Vite-om. Komunicira s backendom jednim POST pozivom (procjena) i jednim opcionalnim GET pozivom (prognoza). Layout koristi Tailwind CSS v4 i shadcn/ui komponente.

src/models/ | src/constants/

`models/delay.ts` definira `DelayRequest`, `DelayResponse` i `WeatherResponse` kao čiste TypeScript interfejsse koji ogledaju Pydantic modele polje po polje. `constants/flight.ts` eksportuje `FIELD_STATUS_CFG`, `DEFAULT_FORM` te nazive dana i mjeseci. `constants/delay.tsx` eksportuje `DELAY_CONFIG`: `Record<number, DelayCfg>` s vizualnom konfiguracijom za klase 1-4. `constants/airlines.ts` i `constants/airports.ts` sadrže objekte `{value, label}` (aerodromi i lat/lon) sortirane abecedno.

src/lib/

`flight.ts` eksportuje `getFieldStatus(name, value)` - switch po nazivu polja s avijacijskim pragovima; `routeDistance()` računa ortodromsku udaljenost u kilometrima; `dateParts()` pretvara datum u mjesec i dan u sedmici (1 = ponedjeljak). `print-report.ts` eksportuje `buildPrintHtml(flightNumber, flightDate, form, result): string` koji generira self-contained A4 HTML dokument s inline CSS-om, gridom podataka o letu i vremenu, kutijom rezultata i linijama za potpis; otvara se putem `window.open()`.

src/api/delay.ts

`submitDelay(data: DelayRequest): Promise<DelayResponse>` šalje post na `/api/delay`;
`fetchWeather(airport, date, hour)` šalje get na `/api/weather`.

src/components/FlightForm.tsx

Root komponenta, vlasnik stanja: `form`, `result`, `flightNumber`, `flightDate`, `isLoading`, `error`.
`handleAirportChange` automatski popunjava udaljenost; `handleFetchWeather` puni vremenska polja iz prognoze; `handleSubmit` validira obavezna polja, izvodi mjesec/dan iz datuma i poziva API; `handleReset` vraća `DEFAULT_FORM`; `handlePrint` poziva `buildPrintHtml`.

src/components/flight/

`WeatherField.tsx` prikazuje numerički input s live status bedžom; koristi `type="text"` s lokalnim stanjima `focused` i `draft` kako bi izbjegao problem vodećih nula, uz opcionalnu podršku za negativne vrijednosti (temperatura). `DepartureHourSlider.tsx` renderira range input 0-23 s CSS linear-gradient track fillom i verbalnom labelom (Noć / Rano jutro / Prije podne / Popodne / Veče). `SearchSelect.tsx` je generički searchable combobox (aviokompanija, polazni i dolazni aerodrom) koji razlikuje query string i odabrani model string; `onMouseDown` sprečava prerano zatvaranje dropdowna. `DelayResult.tsx` prikazuje bedž klase, preporuku i distribuciju vjerovatnoće kao progress barove bojane prema `DELAY_CONFIG`. `HelpPanel.tsx` je sklopivi panel s referentnim opsezima za sva polja i legendom boja.

3. Tok podataka i horizontalni aspekti

End-to-end tok zahtjeva

`handleSubmit()` validira odabire → `dateParts(flightDate)` → `submitDelay(payload)` šalje post zahtjev → FastAPI Pydantic validacija → `predict_delay(): _load_model()` (keširano), `Pipeline.predict() + predict_proba()`, `DelayResult` dataclass → `DelayResponse` JSON → `setResult(res)` → `DelayResult` komponenta. TypeScript interfejsi u `src/models/` i Pydantic modeli u `app/routes/delay.py` moraju biti ručno usklađeni jer nema generisanja koda.

Pragovi vremenskih uslova

`getFieldStatus()` implementira dvostruku klasifikaciju isključivo za vizualni UI feedback - ne utiče na ML predikciju.

Polje	Opseg upozorenja	Kritični opseg
temperature	< -5 ili > 35 °C	< -15 ili > 40 °C
precipitation	> 1 mm/h	> 5 mm/h
snowfall	> 0.1 cm/h	> 1 cm/h
wind_speed	> 30 km/h	> 50 km/h
cloud_cover	> 85 %	-

Generisanje print izvještaja

`buildPrintHtml()` producira self-contained HTML s inline CSS-om: ljepljivi toolbar s dugmadima za print/zatvaranje, A4 kartica, grid podataka o letu i vremenu, kutija rezultata, probability barovi i linije za potpis. Kodovi se razrješavaju kroz `AIRLINES.find()` / `AIRPORTS.find()` kako bi se prikazivalo "Delta Air Lines" umjesto "DL".

Poznati nedostaci i prijedlozi unapređenja

Nedostatak	Uticaj	Prijedlog ispravke
Vrijeme samo na polaznom aerodromu	Propušta kašnjenja uzrokovana vremenom na dolasku	Dodati vremenske varijable za dolazni aerodrom
Neuravnotežene klase (~78% na vrijeme)	Niži recall za manjinske klase	Jače ponderisanje, SMOTE ili prag odluke po klasi
Ručno retreniranje	Model zastari	CI job ili zakazana skripta (novi BTS mjeseci)

Zaključak

Stratus predstavlja funkcionalan prototip sistema za procjenu kašnjenja letova koji uspješno integriše moderne web tehnologije s mašinskim učenjem i vanjskim izvorima podataka. Trodijelna arhitektura - React frontend, FastAPI backend i XGBoost model - omogućava jasno razdvajanje odgovornosti i relativno jednostavno proširivanje sistema. Model treniran na 4,2 miliona stvarnih letova i satnim vremenskim podacima postiže tačnost od ~75,7% na testnom skupu. Validacija na Pydantic i TypeScript nivou osigurava integritet podataka kroz cijeli stack, dok lazy loading i keširanje modela doprinose efikasnosti pri zaključivanju. Identificirani nedostaci, posebno izostanak vremenskih uslova na dolaznom aerodromu, prioriteta su stavka za sljedeću iteraciju.